



Bangladesh University of Business and Technology

Lab report no : 2

Lab topic: Python Pandas Dataframe and Series

Course name :Data Mining Lab

Course code: CSE 476

Submitted by

Name: Md.Al-Amin

Id: 22234103066

Section : 5

Intake : 50

Submitted to

Md.Aminul Islam Rafi

Lecturer

Dept of CSE ,BUBT

Python Pandas Dataframe

A DataFrame in Python's pandas library is a two-dimensional labeled data structure — basically a table of data with rows and columns.

Creating empty dataframe:

Creating an empty DataFrame in pandas simply means making a DataFrame with no data — no rows and no columns — using the `pd.DataFrame()` constructor without passing any arguments.

```
import pandas as pd
df = pd.DataFrame()
print(df)
```

```
Empty DataFrame
Columns: []
Index: []
```

Creating a DataFrame from a List

In pandas, a Python list can be passed directly to the `pd.DataFrame()` constructor to create a DataFrame.

- A single list → becomes a single-column DataFrame.
- A list of lists → becomes a DataFrame with multiple columns.

```
import pandas as pd
data = [1,2,3,4,5]
df = pd.DataFrame(data)
print(df)
```

```
0
0  1
1  2
2  3
3  4
4  5
```

Here is another example of creating a Pandas DataFrame from the Python list of list.

```
import pandas as pd
data = [['Alamin',24],['Hasib',12],['Robert',13]]
df = pd.DataFrame(data,columns=['Name','Age'])
print(df)
```

	Name	Age
0	Alamin	24
1	Hasib	12
2	Robert	13

Creating a DataFrame from a Dictionary of ndarrays or Lists

In pandas, a DataFrame can be created from a dictionary where:

- Keys → become column names.
- Values → are lists or NumPy arrays of the same length.
- If no index is specified → pandas creates a default index 0, 1, 2, ..
- If an index is specified → the index length must match the data length.

creating the DataFrame from a Python dictionary

```
import pandas as pd
data = {'Name':['Alamin', 'Jack', 'Steve', 'Ricky'],'Age':[28,34,29,42]}
df = pd.DataFrame(data)
print(df)
```

	Name	Age
0	Alamin	28
1	Jack	34
2	Steve	29
3	Ricky	42

create an indexed DataFrame using arrays

```
import pandas as pd
data = {'Name': ['Alamin', 'Jack', 'Steve', 'Ricky'], 'Age': [28, 34, 29, 42]}
df = pd.DataFrame(data, index=['indx1', 'indx2', 'indx3', 'indx4'])
print(df)
```

	Name	Age
indx1	Alamin	28
indx2	Jack	34
indx3	Steve	29
indx4	Ricky	42

Creating a DataFrame from a List of Dictionaries

- Each dictionary in the list represents one row of data.
- The keys in the dictionaries become the column names automatically.
- Missing keys in some dictionaries will result in NaN (Not a Number) in those positions

create a DataFrame by passing a list of dictionaries:

```
import pandas as pd
data = [{'a': 1, 'b': 2}, {'a': 5, 'b': 10, 'c': 20}]
df = pd.DataFrame(data)
print(df)
```

	a	b	c
0	1	2	NaN
1	5	10	20.0

create a DataFrame with a list of dictionaries, row indices, and column indices

```
import pandas as pd
data = [{'a': 1, 'b': 2}, {'a': 5, 'b': 10, 'c': 20}]

#With two column indices, values same as dictionary keys
df1 = pd.DataFrame(data, index=['first', 'second'], columns=['a', 'b'])

#With two column indices with one index with other name
df2 = pd.DataFrame(data, index=['first', 'second'], columns=['a', 'b1'])
print(df1)
print(df2)
```

	a	b
first	1	2
second	5	10

	a	b1
first	1	NaN
second	5	NaN

Create a DataFrame from Dict of Series

Dictionary of Series can be passed to form a DataFrame. The resultant index is the union of all the series indexes passed.

```
import pandas as pd

d = {'one' : pd.Series([1, 2, 3], index=['a', 'b', 'c']),
     'two' : pd.Series([1, 2, 3, 4], index=['a', 'b', 'c', 'd'])}

df = pd.DataFrame(d)
print(df)
```

	one	two
a	1.0	1
b	2.0	2
c	3.0	3
d	NaN	4

Another example of creating a Pandas DataFrame from a Series

```
import pandas as pd

data = pd.Series([1, 2, 3, 4], index=['a', 'b', 'c', 'd'])

df = pd.DataFrame(data)
print(df)
```

```
0
a  1
b  2
c  3
d  4
```

Accessing the DataFrame Rows Labels

The index attribute in Pandas is used to access row labels in a DataFrame. It returns an index object containing the series of labels corresponding to the data represented in the each row of the DataFrame. These labels can be integers, strings, or other hashable types

```
import pandas as pd

# Create a DataFrame
df = pd.DataFrame({
    'Name': ['Steve', 'Lia', 'Vin', 'Katie'],
    'Age': [32, 28, 45, 38],
    'Gender': ['Male', 'Female', 'Male', 'Female'],
    'Rating': [3.45, 4.6, 3.9, 2.78]},
    index=['r1', 'r2', 'r3', 'r4'])
```

```
# Access the rows of the DataFrame
result = df.index
print('Output Accessed Row Labels:', result)
```

```
Output Accessed Row Labels: Index(['r1', 'r2', 'r3', 'r4'], dtype='object')
```

Modifying DataFrame Row Labels

With the index attribute you can also modify the row labels of a DataFrame.

Example

Here is an example that demonstrates accessing and modifying the row labels of the Pandas DataFrame using the index attribute.

```
import pandas as pd

# Create a DataFrame
df = pd.DataFrame({
    'Name': ['Steve', 'Lia', 'Vin', 'Katie'],
    'Age': [32, 28, 45, 38],
    'Gender': ['Male', 'Female', 'Male', 'Female'],
    'Rating': [3.45, 4.6, 3.9, 2.78]},
    index=['r1', 'r2', 'r3', 'r4'])

# Display the Input DataFrame
print('Input DataFrame:\n', df)

# Modify the Row Labels of the DataFrame
df.index = [100, 200, 300, 400]
print('Output Modified DataFrame with the updated index labels:\n', df)
```

Input DataFrame:

	Name	Age	Gender	Rating
r1	Steve	32	Male	3.45
r2	Lia	28	Female	4.60
r3	Vin	45	Male	3.90
r4	Katie	38	Female	2.78

Output Modified DataFrame with the updated index labels:

	Name	Age	Gender	Rating
100	Steve	32	Male	3.45
200	Lia	28	Female	4.60
300	Vin	45	Male	3.90
400	Katie	38	Female	2.78

Accessing The DataFrame Columns Labels

The Pandas `pd.columns` attribute is used to access the labels of the columns in the DataFrame. You can access and modify these column labels similarly to how we work with row labels.

Example

The following example demonstrates how to access the DataFrame column labels using the `pd.columns` attribute

```
import pandas as pd

# Create a DataFrame
df = pd.DataFrame({
    'Name': ['Steve', 'Lia', 'Vin', 'Katie'],
    'Age': [32, 28, 45, 38],
    'Gender': ['Male', 'Female', 'Male', 'Female'],
    'Rating': [3.45, 4.6, 3.9, 2.78]},
    index=['r1', 'r2', 'r3', 'r4'])
```

```
# Access the column labels of the DataFrame
result = df.columns
print('Output Accessed column Labels:', result)
```

```
Output Accessed column Labels: Index(['Name', 'Age', 'Gender', 'Rating'], dtype='object')
```

Modifying the DataFrame Column Labels

Column labels can be modified using the columns attribute.

Example

This example demonstrates how to access and modify the DataFrame column labels using the pd.columns attribute.

```
import pandas as pd

# Create a DataFrame
df = pd.DataFrame({
    'Name': ['Steve', 'Lia', 'Vin', 'Katie'],
    'Age': [32, 28, 45, 38],
    'Gender': ['Male', 'Female', 'Male', 'Female'],
    'Rating': [3.45, 4.6, 3.9, 2.78]},
    index=['r1', 'r2', 'r3', 'r4'])

# Display the Input DataFrame
print('Input DataFrame:\n', df)

# Modify the Column Labels of the DataFrame
df.columns = ['Col1', 'Col2', 'Col3', 'Col4']
print('Output Modified DataFrame with the updated Column Labels\n:', df)
```

Input DataFrame:

	Name	Age	Gender	Rating
r1	Steve	32	Male	3.45
r2	Lia	28	Female	4.60
r3	Vin	45	Male	3.90
r4	Katie	38	Female	2.78

Output Modified DataFrame with the updated Column Labels

```
:      Col1  Col2   Col3  Col4
r1  Steve    32    Male  3.45
r2   Lia     28  Female  4.60
r3   Vin     45    Male  3.90
r4  Katie     38  Female  2.78
```

Python Pandas - Slicing a DataFrame Object

Pandas DataFrame slicing is a process of extracting specific rows, columns, or subsets of data based on both position and labels. DataFrame slicing is a common operation while working with large datasets, it is similar to Python lists and NumPy ndarrays, DataFrame slicing uses the [] operator and specific slicing attributes like .iloc[] and .loc[] to retrieve data efficiently.

Introduction to Pandas DataFrame Slicing

Pandas DataFrame slicing is performed using two main attributes, which are –

.iloc[]: For slicing based on position (integer-based indexing).

.loc[]: For slicing based on labels (index labels or column labels).

Slicing DataFrame Rows by Position

The following example demonstrates how to slice the DataFrame rows using the DataFrame.iloc[] attribute.

```
import pandas as pd

# Create a Pandas DataFrame
df = pd.DataFrame([[ 'a', 'b'], [ 'c', 'd'], [ 'e', 'f'], [ 'g', 'h']], columns=[ 'col1', 'col2'])

# Display the DataFrame
print("Input DataFrame:")
print(df)

# Slice rows based on position
result = df.iloc[1:3, :]
print("Output:")
print(result)
```

Input DataFrame:

	col1	col2
0	a	b
1	c	d
2	e	f
3	g	h

Output:

	col1	col2
1	c	d
2	e	f

Slicing DataFrame Rows and Columns using .loc[]

The following example demonstrates how to slice a DataFrame rows and columns by using their labels with the .loc[] attribute.

```
import pandas as pd

# Create a DataFrame with labeled indices
df = pd.DataFrame([[ 'a','b'], [ 'c','d'], [ 'e','f'], [ 'g','h']], columns=[ 'col1', 'col2'], index=[ 'r1', 'r2', 'r3', 'r4'])

# Display the DataFrame
print("Original DataFrame:")
print(df)

# Slice rows and columns by label
result = df.loc[ 'r1': 'r3', 'col1']
print("Output:")
print(result)
```

```
Original DataFrame:
  col1 col2
r1    a    b
r2    c    d
r3    e    f
r4    g    h
Output:
r1    a
r2    c
r3    e
Name: col1, dtype: object
```

DataFrame Column Slicing

Similar to the above row slicing, Pandas DataFrame Column slicing can also be done using the `.iloc[]` for position and `.loc[]` for labels.

Example: Column Slicing using `iloc[]`

The following example slices the DataFrame columns based on their integer position

```
import pandas as pd

data = {'A': [1, 2, 3], 'B': [4, 5, 6], 'C': [7, 8, 9]}
df = pd.DataFrame(data)

# Slice a single column
col_A = df.iloc[:, 0]
print("Slicing a single column A using iloc[:]")
print(col_A)

# Slice multiple columns
cols_AB = df.iloc[:, 0:2]
print("Slicing multiple columns A and B using iloc[:]")
print(cols_AB)
```

Slicing a single column A using `iloc[]`:

```
0    1
1    2
2    3
```

Name: A, dtype: int64

Slicing multiple columns A and B using `iloc[]`:

```
   A  B
0  1  4
1  2  5
2  3  6
```

Example: Column Slicing Using loc[]

```
import pandas as pd

data = {'A': [1, 2, 3], 'B': [4, 5, 6], 'C': [7, 8, 9]}
df = pd.DataFrame(data)

# Slice a single column by label
col_A = df.loc[:, 'A']
print("Slicing a single column A using loc[]:")
print(col_A)

# Slice multiple columns by label
cols_AB = df.loc[:, 'A':'B']
print("Slicing Multiple columns A and B using loc[]:")
print(cols_AB)
```

Slicing a single column A using loc[]:

```
0    1
1    2
2    3
```

Name: A, dtype: int64

Slicing Multiple columns A and B using loc[]:

```
   A  B
0  1  4
1  2  5
2  3  6
```

Python Pandas - Modifying DataFrame

Pandas DataFrame is two-dimensional data structure that can be used for storing and manipulating tabular data. It consists of rows and columns making it similar to a spreadsheet or SQL table. Modifying a Pandas DataFrame is a crucial step in data preprocessing, data analysis, and data cleaning. Some of the most common DataFrame modifications include –

Renaming column or row labels.

Adding or inserting new columns.

Updating or replacing existing column values.

Removing unnecessary columns.

Renaming Column Labels in a DataFrame

Renaming column or row labels improves data readability and helps standardize column names. The rename() method in Pandas allows renaming one or more columns or row labels. The following example uses the DataFrame.rename() method to rename a columns name of a DataFrame.

```
import pandas as pd

# Create a DataFrame
df = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})

# Display original DataFrame
print("Original DataFrame:")
print(df)

# Rename column 'A' to 'aa'
df = df.rename(columns={'A': 'aa'})

# Display modified DataFrame
print("Modified DataFrame:")
print(df)
```

Original DataFrame:

	A	B
0	1	4
1	2	5
2	3	6

Modified DataFrame:

	aa	B
0	1	4
1	2	5
2	3	6

Renaming Row Labels in a DataFrame

Similarly, you can rename row labels using of a Pandas DataFrame using the index parameter of the rename() method.

Example

This example demonstrates how to rename the row labels of a Pandas DataFrame using the rename(index={ }) method.

```
import pandas as pd

# Create a DataFrame
df = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]}, index=['x', 'y', 'z'])

# Display original DataFrame
print("Original DataFrame:")
print(df)

# Rename the multiple row labels
df = df.rename(index={'x': 'r1', 'y': 'r2', 'z': 'r3'})

# Display modified DataFrame
print("Modified DataFrame:")
print(df)
```

Original DataFrame:

	A	B
x	1	4
y	2	5
z	3	6

Modified DataFrame:

	A	B
r1	1	4
r2	2	5
r3	3	6

Adding or Inserting Columns in a DataFrame

Adding a new column to an existing DataFrame is straightforward. The simplest way is to directly assign values to the DataFrame using a new column name. Additionally, you can use the `DataFrame.insert()` method to insert a new column at a specified location.

Example: Adding a New Column Directly

The following example demonstrates how to add a new column directly to a DataFrame.

```
import pandas as pd

# Create a DataFrame
df = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})

# Add a new column 'C' with values
df['C'] = [7, 8, 9]

# Display updated DataFrame
print("DataFrame after adding a new column 'C':")
print(df)
```

DataFrame after adding a new column 'C':

	A	B	C
0	1	4	7
1	2	5	8
2	3	6	9

Example: Inserting a Column at a Specific Position

This example demonstrates how to insert a column at a specific index of a DataFrame using the `DataFrame.insert()` method. In this example we will insert the columns D at index position 1.

```
import pandas as pd

# Create a DataFrame
df = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})

# Add a new column 'C' with values
df['C'] = [7, 8, 9]

# Display updated DataFrame
print("DataFrame after adding a new column 'C':")
print(df)
```

DataFrame after adding a new column 'C':

	A	B	C
0	1	4	7
1	2	5	8
2	3	6	9

```
import pandas as pd

# Create a DataFrame
df = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})

# Insert a new column 'D' at position 1
df.insert(1, 'D', [10, 11, 12])

# Display updated DataFrame
print("DataFrame after inserting column 'D' at position 1:")
print(df)
```

DataFrame after inserting column 'D' at position 1:

	A	D	B
0	1	10	4
1	2	11	5
2	3	12	6

Replacing the Contents of a DataFrame

Replacing the contents of the DataFrame can be done by multiple ways, one of the easiest way is assigning new values directly to the particular part of the DataFrame.

Example: Replacing a Columns Values

The following example demonstrates how to replace/update particular column values of a DataFrame using the direct assignment way.

```
import pandas as pd

# Create a DataFrame
df = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})

# Insert a new column 'D' at position 1
df.insert(1, 'D', [10, 11, 12])

# Display updated DataFrame
print("DataFrame after inserting column 'D' at position 1:")
print(df)
```

DataFrame after inserting column 'D' at position 1:

	A	D	B
0	1	10	4
1	2	11	5
2	3	12	6

```
import pandas as pd

# Create a DataFrame
df = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})

# Replace the contents of column 'A' with new values
df['A'] = [10, 20, 30]

# Display updated DataFrame
print("DataFrame after replacing column 'A':")
print(df)
```

DataFrame after replacing column 'A':

	A	B
0	10	4
1	20	5
2	30	6

Replacing Specific Values using the replace() method

You can also use the DataFrame.replace() method to replace specific values within a column of a DataFrame.

Example

This example demonstrates how to replace specific values in a DataFrame using the DataFrame.replace() method.

```
import pandas as pd

# Create a DataFrame
df = pd.DataFrame({'A': [1, 2, 3], 'B': [4, 5, 6]})

# Display the Input DataFrame
print("Original DataFrame:", df, sep='\n')

# Replace the contents
df.replace({'A': 1, 'B': 6}, 100, inplace=True)

# Display updated DataFrame
print("DataFrame after replacing column 'A':")
print(df)
```

Original DataFrame:

	A	B
0	1	4
1	2	5
2	3	6

DataFrame after replacing column 'A':

	A	B
0	100	4
1	2	5
2	3	100

Python Pandas - Arithmetic Operations on DataFrame

Operation	Example with Operator	Description
Addition	df + 2	Adds 2 to each element of the DataFrame
Subtraction	df - 2	Subtracts 2 from each element
Multiplication	df * 2	Multiplies each element by 2
Division	df / 2	Divides each element by 2
Exponentiation	df ** 2	Raises each element to the power of 2
Modulus	df % 2	Finds the remainder when divided by 2
Floor Division	df // 2	Divides and floors the quotient

```
import pandas as pd

# Create a sample DataFrame
data = {'A': [1, 2, 3, 4], 'B': [5, 6, 7, 8]}
df = pd.DataFrame(data)

# Display the input DataFrame
print("Input DataFrame:\n", df)

# Perform arithmetic operations
print("\nAddition:\n", df + 2)
print("\nSubtraction:\n", df - 2)
print("\nMultiplication:\n", df * 2)
print("\nDivision:\n", df / 2)
print("\nExponentiation:\n", df ** 2)
print("\nModulus:\n", df % 2)
print("\nFloor Division:\n", df // 2)
```

Input DataFrame:

	A	B
0	1	5
1	2	6
2	3	7
3	4	8

Addition:

	A	B
0	3	7
1	4	8
2	5	9
3	6	10

Subtraction:

	A	B
0	-1	3
1	0	4
2	1	5
3	2	6

Multiplication:

	A	B
0	2	10
1	4	12
2	6	14
3	8	16

Division:

	A	B
0	0.5	2.5
1	1.0	3.0
2	1.5	3.5
3	2.0	4.0

Exponentiation:

	A	B
0	1	25
1	4	36
2	9	49
3	16	64

Modulus:

	A	B
0	1	1
1	0	0
2	1	1
3	0	0

Floor Division:

	A	B
0	0	2
1	1	3
2	1	3
3	2	4

Python Pandas - Series

A Series in Pandas is a one-dimensional labeled array capable of holding data of any type, including integers, floats, strings, and Python objects. It consists of two main components –

Data: The actual values stored in the Series.

Index: The labels or indices that correspond to each data value.

A Series is similar to a one-dimensional ndarray (NumPy array) but with labels, which are also known as indices. These labels can be used to access the data within the Series. By default, the index values are integers starting from 0 to the length of the Series minus one, but you can also manually set the index labels. **Create an Empty Series**

If no data is provided to the Series constructor `pandas.Series()` it will create a basic empty series object.

```
import pandas as pd
s = pd.Series()

# Display the result
print('Resultant Empty Series:\n',s)
```

```
Resultant Empty Series:
Series([], dtype: object)
```

Create a Series from ndarray:

An ndarray is provided as an input data to the Series constructor, then it will create series with that data. If you want specify the custom index then index passed must be of the same length of input data. If no index is specified, then Pandas will automatically generate a default index from starting 0 to length of the input data, i.e., `[0,1,2,3. range(len(array))-1]`.

```
import pandas as pd
import numpy as np
data = np.array(['a', 'b', 'c', 'd'])
s = pd.Series(data)
print(s)
```

```
0    a
1    b
2    c
3    d
dtype: object
```

Create a Series from Python Dictionary

A dictionary can be passed as input to the `pd.Series()` constructor to create a series with the dictionary values. If no index is specified, then the dictionary keys are taken in a sorted order to construct the series index. If index is passed, the values in data corresponding to the labels in the index will be pulled out.

```
#import the pandas library and aliasing as pd
import pandas as pd
import numpy as np
data = {'a' : 0., 'b' : 1., 'c' : 2.}
s = pd.Series(data)
print(s)
```

```
a    0.0
b    1.0
c    2.0
dtype: float64
```

Series object is created with Python dictionary by explicitly specifying the index labels.

```
#import the pandas library and aliasing as pd
import pandas as pd
import numpy as np
data = {'a' : 0., 'b' : 1., 'c' : 2.}
s = pd.Series(data,index=['b','c','x','a'])
print(s)
```

```
b    1.0
c    2.0
x    NaN
a    0.0
dtype: float64
```

Create a Series from Scalar

If you provide a single scalar value as data to the `Pd.Series()` constructor with specified index labels. Then that single value will be repeated to match the length of provided index object.

```
import pandas as pd
import numpy as np
s = pd.Series(5, index=[0, 1, 2, 3])
print(s)
```

```
0    5
1    5
2    5
3    5
dtype: int64
```

Slicing a Series by Position

Pandas Series allows you to select the elements based on their position(i.e, Index values), just like Python list object.Example: Slicing range of values from a Series

```
import pandas as pd
import numpy as np

data = np.array(['a', 'b', 'c', 'd'])
s = pd.Series(data)

# Display the Original series
print('Original Series:',s, sep='\n')

# Slice the range of values
result = s[1:3]

# Display the output
print('Values after slicing the Series:', result, sep='\n')
```

```
Original Series:
0    a
1    b
2    c
3    d
dtype: object
Values after slicing the Series:
1    b
2    c
dtype: object
```


Example: Slicing the First Three Elements from a Series

```
import pandas as pd
s = pd.Series([1,2,3,4,5],index = ['a','b','c','d','e'])

#retrieve the first three element
print(s[:3])
```

a 1
b 2
c 3
dtype: int64

Slicing the Last Three Elements from a Series:

```
import pandas as pd
s = pd.Series([1,2,3,4,5],index = ['a','b','c','d','e'])

#retrieve the last three element
print(s[-3:])
```

c 3
d 4
e 5
dtype: int64

Slicing a Series by Label

A Pandas Series is like a fixed-size Python **dict** in that you can get and set values by index labels.

```
import pandas as pd
s = pd.Series([1,2,3,4,5],index = ['a','b','c','d','e'])

# Slice multiple elements
print(s['a':'d'])
```

a 1
b 2
c 3
d 4
dtype: int64

Slicing First Three Elements using the Labels

```
import pandas as pd
s = pd.Series([1,2,3,4,5],index = ['a','b','c','d','e'])

# Slice multiple elements
print(s['c'])

a    1
b    2
c    3
dtype: int64
```

Modifying Values after Slicing

After slicing a Series, you can also modify the values, by assigning the new values to those sliced elements.

```
import pandas as pd
s = pd.Series([1,2,3,4,5])

# Display the original series
print("Original Series:\n",s)

# Modify the values of first two elements
s[:2] = [100, 200]

print("Series after modifying the first two elements:",s)
```

Original Series:

```
0    1
1    2
2    3
3    4
4    5
```

dtype: int64

Series after modifying the first two elements: 0 100

```
1    200
2     3
3     4
4     5
```

dtype: int64

Arithmetic Operations on a Series with Scalar Value

Operation	Example	Description
Addition	$s + 2$	Adds 2 to each element
Subtraction	$s - 2$	Subtracts 2 from each element
Multiplication	$s * 2$	Multiplies each element by 2
Division	$s / 2$	Divides each element by 2
Exponentiation	$s ** 2$	Raises each element to the power of 2
Modulus	$s \% 2$	Finds remainder when divided by 2
Floor Division	$s // 2$	Divides and floors the quotient

```
import pandas as pd
s = pd.Series([1,2,3,4,5],index = ['a','b','c','d','e'])

# Display the Input Series
print('Input Series\n',s)

# Apply all Arithmetic Operation and Display the Results
print('\nAddition:\n',s+2)
print('\nSubtraction:\n', s-2)
print('\nMultiplication:\n', s * 2)
print('\nDivision:\n', s/2)
print('\nExponentiation:\n', s**2)
print('\nModulus:\n', s%2)
print('\nFloor Division:\n', s//2)
```

Input Series

```
a    1
b    2
c    3
d    4
e    5
dtype: int64
```

Addition:

```
a    3
b    4
c    5
d    6
e    7
dtype: int64
```

Subtraction:

```
a   -1
b    0
c    1
d    2
e    3
dtype: int64
```

Multiplication:

```
a     2
b     4
c     6
d     8
e    10
dtype: int64
```

Division:

```
a    0.5
b    1.0
c    1.5
d    2.0
e    2.5
dtype: float64
```

Exponentiation:

```
a     1
b     4
c     9
d    16
e    25
dtype: int64
```

Modulus:

a	1
b	0
c	1
d	0
e	1

dtype: int64

Floor Division:

a	0
b	1
c	1
d	2
e	2

dtype: int64